

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Донской государственный технический университет»
(ДГТУ)

ОТЧЁТ

по лабораторным работам №1–4

по дисциплине «Компьютерные технологии и базы данных»

Выполнил:

студент УТС-11

Клюшниченко А.Г.

Вариант: 7

Ростов-на-Дону, 2026

Содержание

Лабораторная работа №1. Программирование алгоритмов линейной структуры	4
Цель работы	4
Задание	4
Таблица 1 — Исходные данные варианта 7	4
Блок-схема программы	4
Листинг программы	5
Скриншот результата работы программы	6
Вывод	6
Лабораторная работа №2. Программирование алгоритмов разветвляющейся структуры	7
Цель работы	7
Задание	7
Таблица 2 — Исходные данные варианта 7	7
Таблица 3 — Тестовые наборы данных	7
Блок-схема программы	7
Листинг программы	8
Скриншот результата работы программы	9
Вывод	9
Лабораторная работа №3. Программирование алгоритмов регулярной циклической структуры	11
Цель работы	11
Задание	11
Таблица 4 — Исходные данные варианта 7 (задания 3.1–3.3)	11
Архитектура программы	11
Листинги заголовочных файлов	12
Листинги файлов реализации	14
Блок-схемы программы	17
Скриншот результата работы программы	20
Вывод	21
Лабораторная работа №4. Одномерный массив	22
Цель работы	22
Задание	22
Таблица 5 — Исходные данные варианта 7	22
Блок-схема программы	22
Листинг программы	23
Скриншот результата работы программы	24

Вывод	24
Приложение 1	25

Лабораторная работа №1. Программирование алгоритмов линейной структуры

Цель работы

Изучить основы программирования алгоритмов линейной структуры на языке C++. Научиться вычислять математические формулы с использованием стандартных функций и обрабатывать ошибочные ситуации.

Задание

Написать программу на языке C++ для вычисления функции в соответствии с вариантом:

- вычислить заданную формулу на основе отдельной функции с передачей параметров;
- предусмотреть обработку деления на ноль;
- вывести результат с округлением до 4 знаков после запятой;
- построить блок-схему к программе.

Таблица 1 — Исходные данные варианта 7

Вариант	Формула для вычислений	x	y
7	$d = \frac{x \cdot e^{xy} + 8 \sin^2 x}{x(x - y)(3x + y)}$	1,002	−0,5321

Ожидаемый результат при тестовых значениях $x = 1\{, \}002$, $y = -0\{, \}5321$: $d = 1\{, \}6480$.

Блок-схема программы

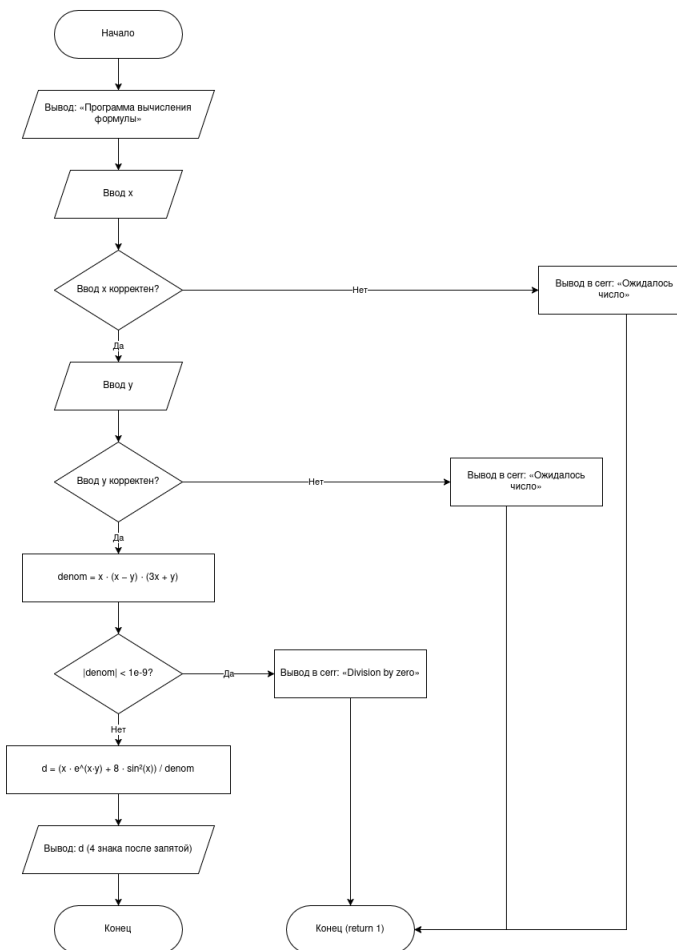


Рис. 1. Блок-схема алгоритма лабораторной работы №1

Листинг программы

Листинг 1 — *main.cpp* (ЛР №1)

```

#include <iostream>
#include <cmath>
#include <iomanip>

double GetD(const double x, const double y) {
    const double denominator = x * (x - y) * (3 * x + y);
    if (std::abs(denominator) < 1e-9) {
        throw std::runtime_error("Division by zero.");
    }

    const double d = (x * exp(x * y) + 8 * sin(x) * sin(x)) /
        denominator;

    return d;
}

int main() {
    //Тестовые значения: x = 1,002; y = -0.5321. Ожидается d = 1,6480.
    std::cout << "A program for calculating a formula.\n";

    double x, y;

```

```

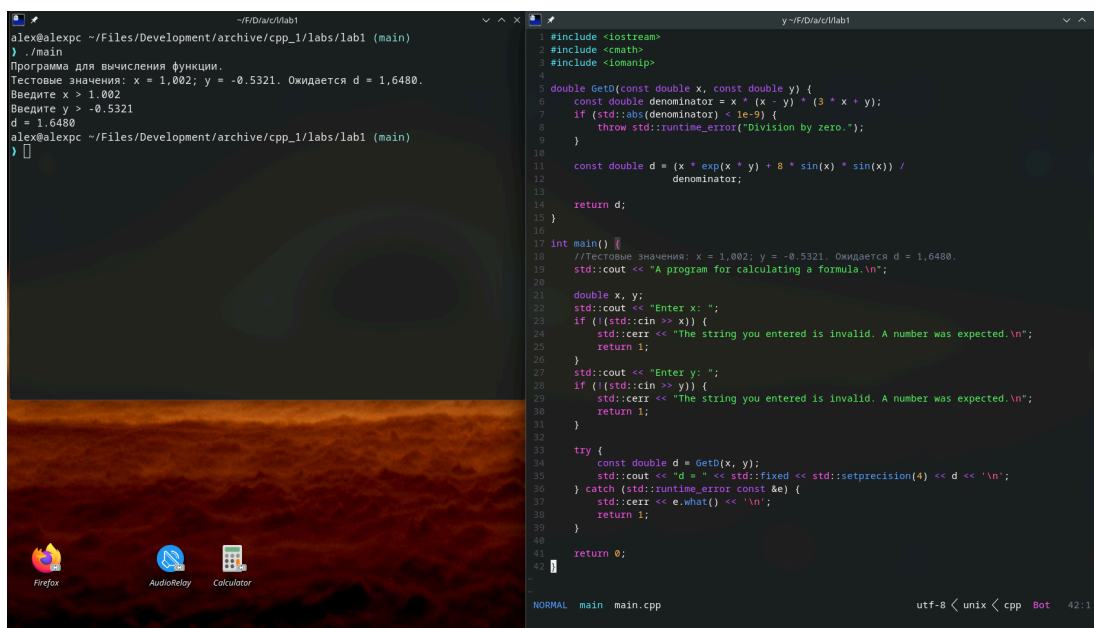
std::cout << "Enter x: ";
if (!(std::cin >> x)) {
    std::cerr << "The string you entered is invalid. A number was expected.\n";
    return 1;
}
std::cout << "Enter y: ";
if (!(std::cin >> y)) {
    std::cerr << "The string you entered is invalid. A number was expected.\n";
    return 1;
}

try {
    const double d = GetD(x, y);
    std::cout << "d = " << std::fixed << std::setprecision(4) << d << '\n';
} catch (std::runtime_error const &e) {
    std::cerr << e.what() << '\n';
    return 1;
}

return 0;
}

```

Скриншот результата работы программы



```

alex@alexpc ~/Files/Development/archive/cpp_1/labs/lab1 (main)
$ ./main
Программа для вычисления функции.
Тестовые значения: x = 1.002; y = -0.5321. Ожидается d = 1.6480.
Введите x > 1.002
Введите y > -0.5321
d = 1.6480
alex@alexpc ~/Files/Development/archive/cpp_1/labs/lab1 (main)
$

```

```

1 #include <iostream>
2 #include <cmath>
3 #include <iomanip>
4
5 double GetD(const double x, const double y) {
6     const double denominator = x * (x - y) * (3 * x + y);
7     if (std::abs(denominator) < 1e-9) {
8         throw std::runtime_error("Division by zero.");
9     }
10    const double d = (x * exp(x * y) + 8 * sin(x) * sin(x)) /
11        denominator;
12    return d;
13 }
14
15 int main() {
16    //Тестовые значения: x = 1.002; y = -0.5321. Ожидается d = 1.6480.
17    std::cout << "A program for calculating a formula.\n";
18
19    double x, y;
20    std::cout << "Enter x: ";
21    if (!(std::cin >> x)) {
22        std::cerr << "The string you entered is invalid. A number was expected.\n";
23        return 1;
24    }
25    std::cout << "Enter y: ";
26    if (!(std::cin >> y)) {
27        std::cerr << "The string you entered is invalid. A number was expected.\n";
28        return 1;
29    }
30
31    try {
32        const double d = GetD(x, y);
33        std::cout << "d = " << std::fixed << std::setprecision(4) << d << '\n';
34    } catch (std::runtime_error const &e) {
35        std::cerr << e.what() << '\n';
36        return 1;
37    }
38
39    return 0;
40 }

```

Рис. 2. Результат выполнения программы лабораторной работы №1

Вывод

В ходе лабораторной работы №1 была разработана программа на языке C++ для вычисления математической формулы линейной структуры. Реализована функция $\text{GetD}(x, y)$, выполняющая вычисление выражения $d = \frac{xe^{xy} + 8\sin^2 x}{x(x-y)(3x+y)}$. Предусмотрена обработка исключительной ситуации деления на ноль. Результат выводится с точностью до четырёх знаков после запятой.

Лабораторная работа №2. Программирование алгоритмов разветвляющейся структуры

Цель работы

Изучить принципы программирования алгоритмов разветвляющейся структуры. Освоить применение условного оператора `if-else` в языке C++ для реализации многоветвевых алгоритмов.

Задание

Написать программу на языке C++ для вычисления функции в соответствии с вариантом:

- объявить тип всех используемых переменных как вещественный;
- предусмотреть ввод значений переменных с клавиатуры;
- подготовить тестовые наборы данных для проверки всех ветвей программы;
- построить блок-схему к программе.

Таблица 2 — Исходные данные варианта 7

Вариант	Формулы и условия реализации
7	$Z = \begin{cases} \max(C; x/C; \sin C) & \text{если } C > 0 \text{ и } x > 2, \\ \min(\min(C; Cx + \sin B); \max(C; B)) & \text{если } C < 0 \text{ и } x > 2, \\ \max(C; B) & \text{иначе} \end{cases}$ Постоянные: $A = 1\{, \}1$; $B = 2\{, \}6$; $D = 4\{, \}2$; $C = \pm 3\{, \}1$

Таблица 3 — Тестовые наборы данных

Набор	C	x	Активная ветвь
1	+3,1	3,0	Ветвь 1: $\max(C, \frac{x}{C}, \sin C)$
2	-3,1	3,0	Ветвь 2: $\min(\min(C, C \cdot x + \sin B), \max(C, B))$
3	+3,1	1,0	Ветвь 3: $\max(C, B)$

Блок-схема программы

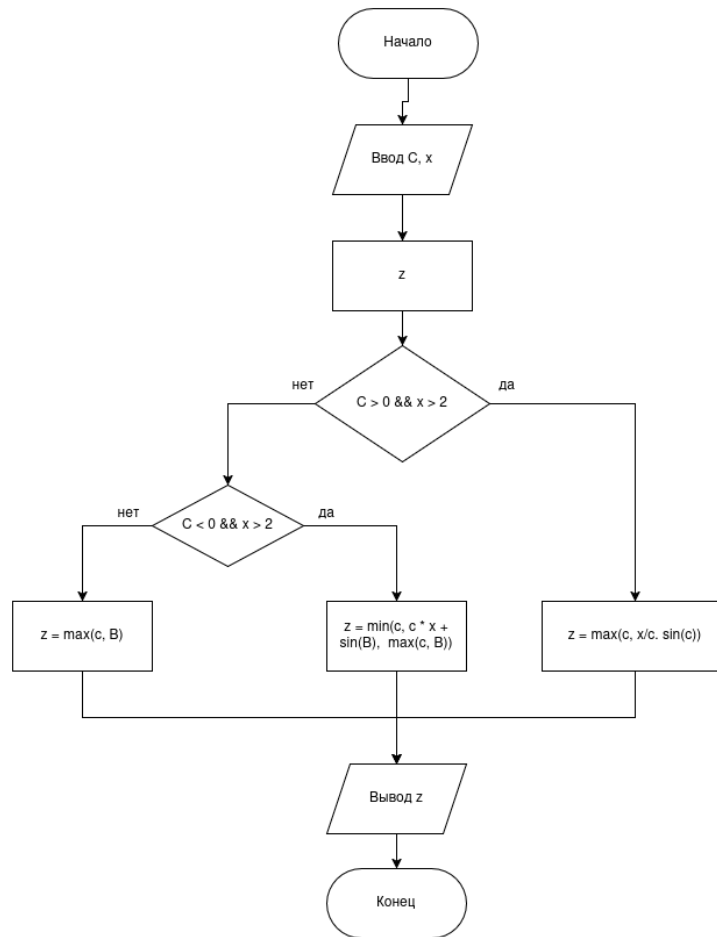


Рис. 3. Блок-схема алгоритма лабораторной работы №2

Листинг программы

Листинг 2 — *main.cpp* (ЛР №2)

```

#include <iostream>
#include <cmath>

constexpr double A = 1.1;
constexpr double B = 2.6;
constexpr double D = 4.2;

int main() {
    std::cout << "A program for calculating a function.\n";

    std::string isVariableCGreaterThanZero;
    std::cout << "Is coefficient C greater than zero? [Y/n] ";
    std::getline(std::cin, isVariableCGreaterThanZero);
    double c;
    if (isVariableCGreaterThanZero.empty() || isVariableCGreaterThanZero == "Y" ||
isVariableCGreaterThanZero == "y") {
        c = 3.1;
    } else {
        c = -3.1;
    }
}

```

```

std::cout << "C = " << c << '\n';

double x;
std::cout << "Enter x: ";
if (!(std::cin >> x)) {
    std::cerr << "The string you entered is invalid. A number was expected.\n";
    return 1;
}

double z;
if (c > 0 && x > 2) {
    z = std::max(std::max(c, x / c), sin(c));
} else if (c < 0 && x > 2) {
    const double t1 = c * x + sin(B);
    const double t2 = std::max(c, B);
    const double t3 = std::min(c, t1);
    z = std::min(t3, t2);
} else {
    z = std::max(c, B);
}
std::cout << "Z = " << z << '\n';

return 0;
}

```

Скриншот результата работы программы

```

alex@alexpc: ~/Files/Development/archive/cpp_1/labs/lab2 (main)
$ ./main
Программа для вычисления функции.
Коэффициент C больше нуля? [Y/n] y
C = 3.1
Введите x: 2
Z = 3.1
alex@alexpc: ~/Files/Development/archive/cpp_1/labs/lab2 (main)
$

```

```

1 #include <iostream>
2 #include <cmath>
3
4 constexpr double A = 1.1;
5 constexpr double B = 2.6;
6 constexpr double D = 4.2;
7
8 int main() {
9     std::cout << "A program for calculating a function.\n";
10
11     std::string isVariableGreaterThanZero;
12     std::cout << "Is coefficient C greater than zero? [Y/n] ";
13     std::getline(std::cin, isVariableGreaterThanZero);
14     double c;
15     if (isVariableGreaterThanZero.empty() || isVariableGreaterThanZero == "Y" || isVariableGreaterThanZero == "y") {
16         c = 3.1;
17     } else {
18         c = -3.1;
19     }
20     std::cout << "C = " << c << '\n';
21
22     double x;
23     std::cout << "Enter x: ";
24     if (!(std::cin >> x)) {
25         std::cerr << "The string you entered is invalid. A number was expected.\n";
26         return 1;
27     }
28
29     double z;
30     if (c > 0 && x > 2) {
31         z = std::max(std::max(c, x / c), sin(c));
32     } else if (c < 0 && x > 2) {
33         const double t1 = c * x + sin(B);
34         const double t2 = std::max(c, B);
35         const double t3 = std::min(c, t1);
36         z = std::min(t3, t2);
37     } else {
38         z = std::max(c, B);
39     }
40     std::cout << "Z = " << z << '\n';
41
42     return 0;
43 }

```

Рис. 4. Результат выполнения программы лабораторной работы №2

Вывод

В ходе лабораторной работы №2 была разработана программа на языке C++ для вычисления функции разветвляющейся структуры. Реализованы три ветви вычислений в зависимости от знака коэффициента C и значения аргумента x . Использованы стандартные функции `std::max`,

`std::min` и `sin`. Программа успешно протестирована на всех ветвях алгоритма.

Лабораторная работа №3. Программирование алгоритмов регулярной циклической структуры

Цель работы

Изучить принципы программирования циклических алгоритмов на языке C++. Освоить операторы цикла `while`, `do-while` и `for`. Реализовать объектно-ориентированную архитектуру с абстрактным базовым классом.

Задание

Написать программы на языке C++ для трёх задач циклической структуры:

- **Задание 3.1** — цикл с предусловием (`while`): табуляция функции, вывод пар (x, y) при $y > 0$;
- **Задание 3.2** — цикл с постусловием (`do-while`): вычисление и вывод значений функции на интервале;
- **Задание 3.3** — цикл со счётчиком (`for`): вычисление суммы ряда.

Таблица 4 — Исходные данные варианта 7 (задания 3.1–3.3)

Задание	Функция / задача	Отрезок	Шаг
3.1	Табуляция и Произведение $Y > 0$: $y = \begin{cases} e^x, & x > 1 \\ 2x-1, & x < 0 \\ -1, & \text{иначе} \end{cases}$	$[-3; 3]$	0,3
3.2	Табуляция функции при $t \in [1; 5]$, $A = 2\{, \}5$, $\Delta t = 0\{, \}5$: $y = \begin{cases} t\sqrt{t-A}, & t > A \\ t \sin(At), & t = A \\ e^{-At} \cos(At), & t < A \end{cases}$	$[1; 5]$	0,5
3.3	Вычислить сумму ряда при введённом x : $f(x) = \sum_{n=3}^8 \frac{e^n x^2}{2n+1}$	—	—

Архитектура программы

Программа построена на основе объектно-ориентированного подхода. Определён абстрактный интерфейс `IFunction` с чистыми виртуальными методами `GetFunctionName()` и `Execute()`. От него наследуются

три класса: `WhileBasedFunction`, `DoWhileBasedFunction` и `ForBasedFunction`. Шаблонная структура `Result<T>` хранит пары (вход, выход). Выбор алгоритма осуществляется через меню с применением полиморфизма.

Листинги заголовочных файлов

Листинг 3 — *utils/Result.h*

```
#pragma once

namespace Laboratory {
    template<typename T>

    struct Result {
        Result(T input, T output) {
            Input = input;
            Output = output;
        }

        T Input;
        T Output;
    };
}
```

Листинг 4 — *utils/CrossPlatformTerminalCleaner.h*

```
#pragma once

#include <iostream>

#ifdef _WIN32
#include <windows.h>
#endif

class CrossPlatformTerminalCleaner {
public:
    static void Clear() {
#ifdef _WIN32
        HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
        CONSOLE_SCREEN_BUFFER_INFO csbi;
        DWORD count;
        DWORD cellCount;
        COORD homeCoords = {0, 0};

        if (hConsole == INVALID_HANDLE_VALUE) return;

        if (!GetConsoleScreenBufferInfo(hConsole, &csbi)) return;
        cellCount = csbi.dwSize.X * csbi.dwSize.Y;

        FillConsoleOutputCharacter(hConsole, ' ', cellCount, homeCoords, &count);
        FillConsoleOutputAttribute(hConsole, csbi.wAttributes, cellCount, homeCoords, &count);
        SetConsoleCursorPosition(hConsole, homeCoords);
#else
        std::cout << "\033[2J\033[1;1H";
#endif
    }
};
```

Листинг 5 — Laboratory/IFunction.h

```
#pragma once
#include <string>
#include <vector>

#include "utils/Result.h"

namespace Laboratory {
    class IFunction {
    public:
        virtual ~IFunction() = default;

        [[nodiscard]] virtual std::string GetFunctionName() const noexcept = 0;
        [[nodiscard]] virtual std::vector<Result<double>> Execute() const = 0 ;
    };
}
```

Листинг 6 — Laboratory/WhileBasedFunction.h

```
#pragma once
#include "IFunction.h"

namespace Laboratory {
    class WhileBasedFunction final : public IFunction {
    public:
        [[nodiscard]] std::string GetFunctionName() const noexcept override {
            return "Exercise 3.1 (using a while loop)";
        }

        [[nodiscard]] std::vector<Result<double> > Execute() const noexcept override;

    private:
        static constexpr double A = -3.0;
        static constexpr double B = 3.0;
        static constexpr double Step = 0.3;

        [[nodiscard]] static std::vector<Result<double> > Compute();
    };
}
```

Листинг 7 — Laboratory/DoWhileBasedFunction.h

```
#pragma once
#include "IFunction.h"

namespace Laboratory {
    class DoWhileBasedFunction final : public IFunction {
    public:
        [[nodiscard]] std::string GetFunctionName() const noexcept override {
            return "Exercise 3.2 (using the do...while loop)";
        }

        [[nodiscard]] std::vector<Result<double> > Execute() const noexcept override;

    private:
        static constexpr double A = 2.5;
        static constexpr double DeltaT = 0.5;
    };
}
```

```

        [[nodiscard]] static std::vector<Result<double> > Compute();
    };
}

```

Листинг 8 — Laboratory/ForBasedFunction.h

```

#pragma once
#include "IFunction.h"

namespace Laboratory {
    class ForBasedFunction final : public IFunction {
    public:
        [[nodiscard]] std::string GetFunctionName() const noexcept override {
            return "Exercise 3.3 (using `for`)";
        }

        [[nodiscard]] std::vector<Result<double> > Execute() const override;

    private:
        [[nodiscard]] static std::vector<Result<double> > Compute(double input);
    };
}

```

Листинги файлов реализации

Листинг 9 — src/main.cpp

```

#include <limits>
#include <memory>

#include "../include/Laboratory/IFunction.h"
#include "../include/Laboratory/WhileBasedFunction.h"
#include "../include/Laboratory/DoWhileBasedFunction.h"
#include "../include/Laboratory/ForBasedFunction.h"
#include "../include/Laboratory/utils/CrossPlatformTerminalCleaner.h"

constexpr int LEFT_OFFSET = 3;

void WaitForEnter() {
    std::cout << "Press Enter to continue..." << std::flush;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::cin.get();
}

void DisplayResult(const std::vector<Laboratory::Result<double> > &results) {
    std::cout << std::string(LEFT_OFFSET, ' ') << "(x, y)\n";

    for (const auto &result: results) {
        std::cout << std::string(LEFT_OFFSET, ' ') << '(' << result.Input << ", " <<
result.Output << ")\n";
    }
}

void DisplayMenu(const std::vector<std::unique_ptr<Laboratory::IFunction> > &functions) {
    std::cout << "Lab Assignment 3.\n\nAvailable options:\n";

    const auto offset = std::string(LEFT_OFFSET, ' ');
}

```

```

    for (size_t i = 0; i < functions.size(); ++i) {
        std::cout << offset << i + 1 << ". " << functions[i]->GetFunctionName() << "\n";
    }
    std::cout << offset << "0. Exit\n";
}

int main() {
    using namespace Laboratory;
    using std::vector, std::unique_ptr;

    vector<unique_ptr<IFunction>> functions;
    functions.push_back(std::make_unique<WhileBasedFunction>());
    functions.push_back(std::make_unique<DoWhileBasedFunction>());
    functions.push_back(std::make_unique<ForBasedFunction>());

    CrossPlatformTerminalCleaner::Clear();
    while (true) {
        DisplayMenu(functions);

        std::cout << "\nSelect an option: ";

        std::string input;
        std::cin >> input;

        if (input == "0") {
            break;
        }

        const IFunction *selected = nullptr;
        for (size_t i = 0; i < functions.size(); i++) {
            if (input == std::to_string(i + 1)) {
                selected = functions[i].get();
                break;
            }
        }

        if (!selected) {
            std::cout << "There is no such option.\n";
            continue;
        }

        CrossPlatformTerminalCleaner::Clear();
        std::cout << selected->GetFunctionName() << '\n';
        auto results = selected->Execute();

        DisplayResult(results);
        WaitForEnter();
        CrossPlatformTerminalCleaner::Clear();
    }

    return 0;
}

```

Листинг 10 — *src/WhileBasedFunction.cpp* (задание 3.1)

```

#include <vector>
#include <cmath>
#include "../include/Laboratory/WhileBasedFunction.h"
#include "../include/Laboratory/utils/Result.h"

```

```

namespace Laboratory {
    std::vector<Result<double> > WhileBasedFunction::Execute() const noexcept {
        return Compute();
    }

    std::vector<Result<double>> WhileBasedFunction::Compute() {
        auto result = std::vector<Result<double> >();

        double x = A;
        double y;
        while (x < B) {
            if (x > 1) {
                y = std::exp(x);
            } else if (x < 0) {
                y = 2 * x - 1;
            } else {
                y = -1;
            }

            if (y > 0) {
                result.emplace_back(x, y);
            }

            x += Step;
        }

        return result;
    }
}

```

Листинг 11 — *src/DoWhileBasedFunction.cpp* (задание 3.2)

```

#include <vector>
#include <cmath>
#include "../include/Laboratory/DoWhileBasedFunction.h"
#include "../include/Laboratory/utils/Result.h"

namespace Laboratory {
    std::vector<Result<double> > DoWhileBasedFunction::Execute() const noexcept {
        return Compute();
    }

    std::vector<Result<double>> DoWhileBasedFunction::Compute() {
        auto result = std::vector<Result<double> >();

        double t = 1;
        double y;
        do {
            if (t > A) {
                y = t * sqrt(t - A);
            }
            else if (std::abs(t - A) < 1e-9) {
                y = t * sin(A * t);
            }
            else if (t < A) {
                y = exp(-A * t) * cos(A * t);
            }

            result.emplace_back(t, y);
            t += DeltaT;
        } while (t <= 5);
    }
}

```

```

        return result;
    }
}

```

Листинг 12 — src/ForBasedFunction.cpp (задание 3.3)

```

#include <cmath>
#include <iostream>
#include <stdexcept>
#include <vector>

#include "../include/Laboratory/ForBasedFunction.h"
#include "../include/Laboratory/Utils/Result.h"

namespace Laboratory {
    std::vector<Result<double> > ForBasedFunction::Compute(double input) {
        std::vector<Result<double> > result;

        double sum = 0;
        for (int n = 3; n <= 8; n++) {
            sum += std::exp(static_cast<double>(n)) * input * input
                / static_cast<double>(2 * n + 1);
        }

        result.emplace_back(input, sum);
        return result;
    }

    std::vector<Result<double> > ForBasedFunction::Execute() const {
        std::cout << "Enter x: ";
        double x;

        if (!(std::cin >> x)) {
            const std::string message =
                "The string you entered is invalid. A number was expected.";
            std::cerr << message << '\n';
            throw std::runtime_error(message);
        }

        return Compute(x);
    }
}

```

Блок-схемы программы

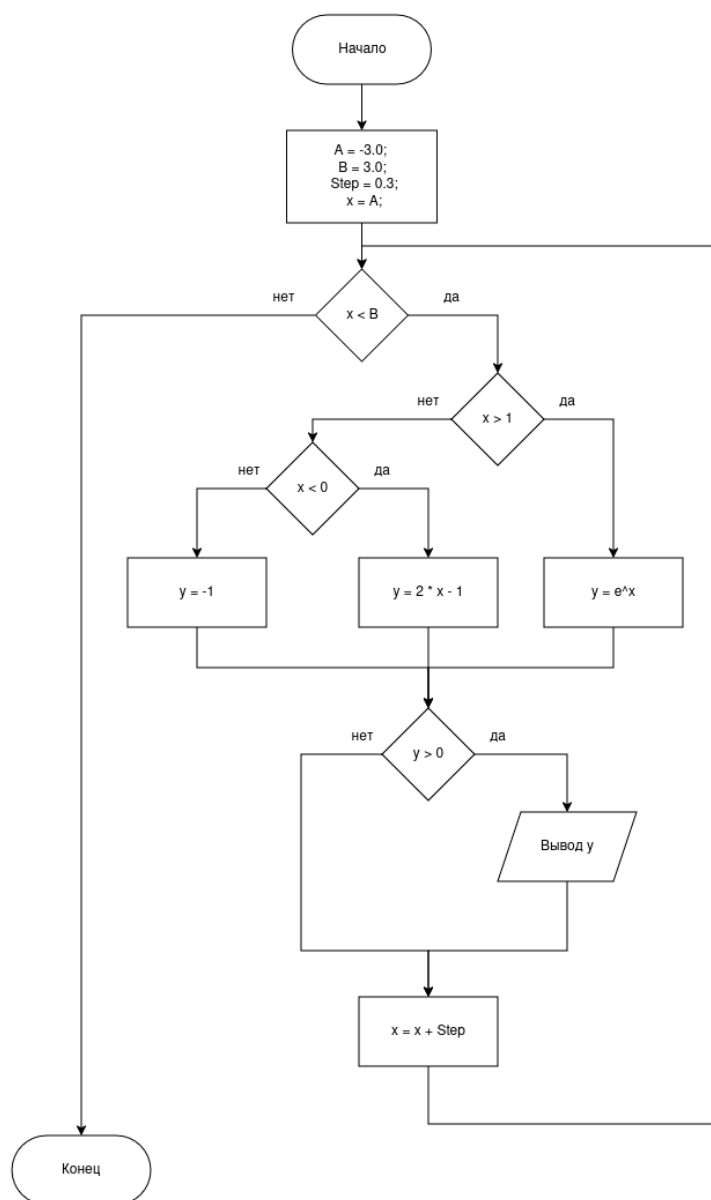


Рис. 5. Блок-схема алгоритма задания 3.1 (цикл while)

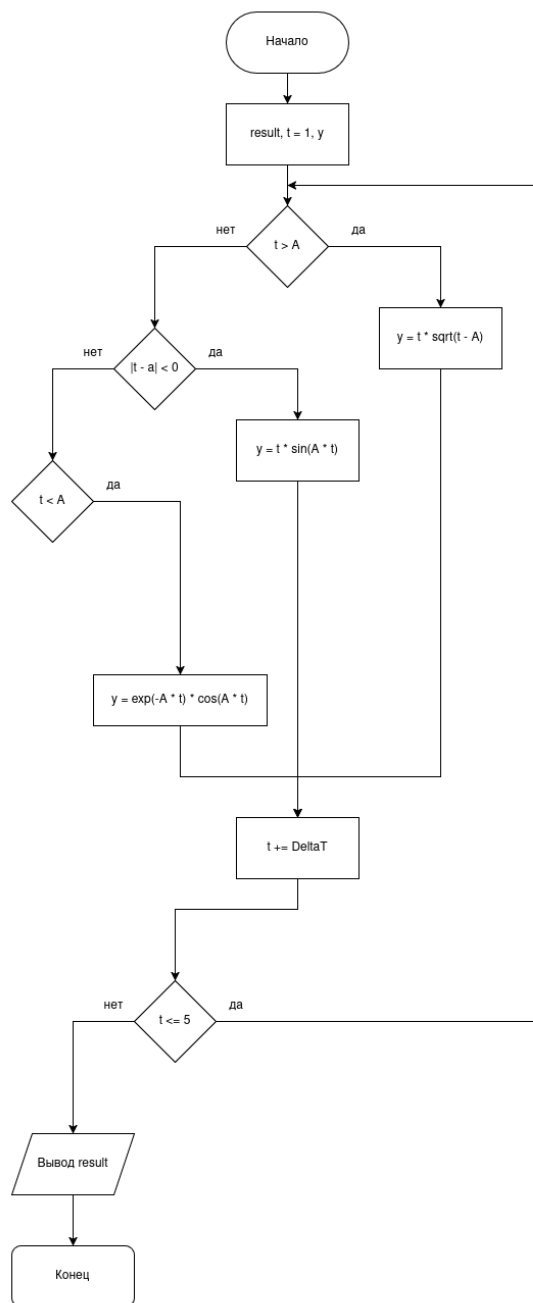


Рис. 6. Блок-схема алгоритма задания 3.2 (цикл do-while)

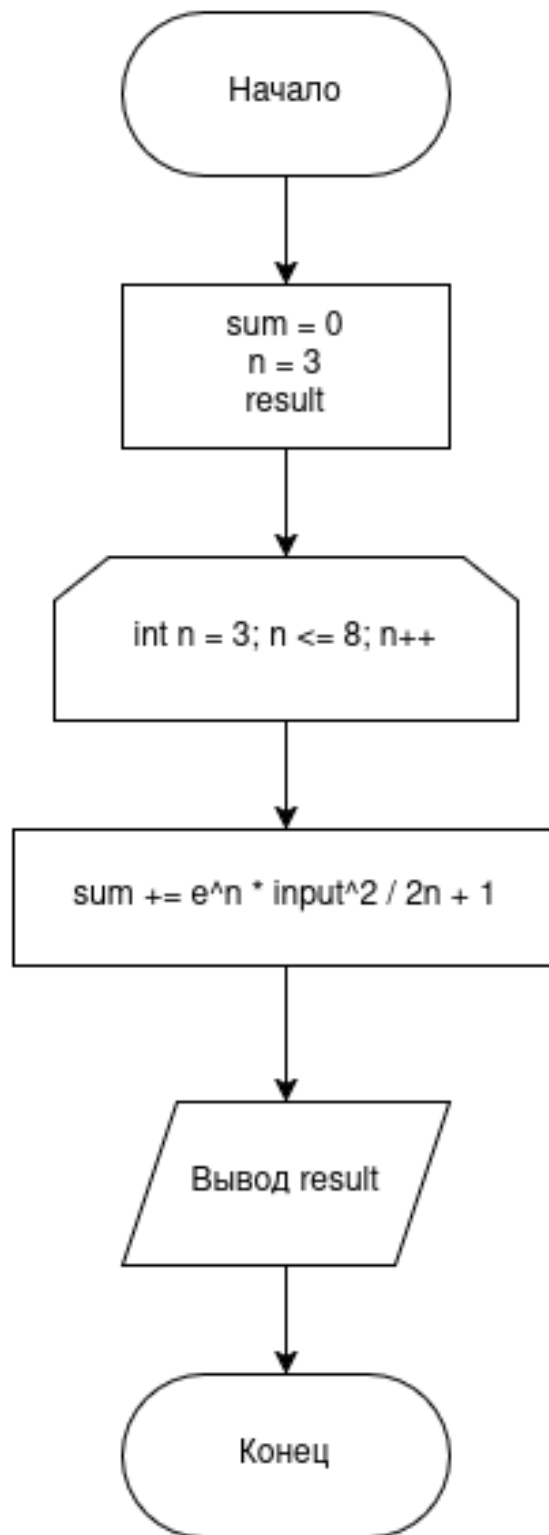
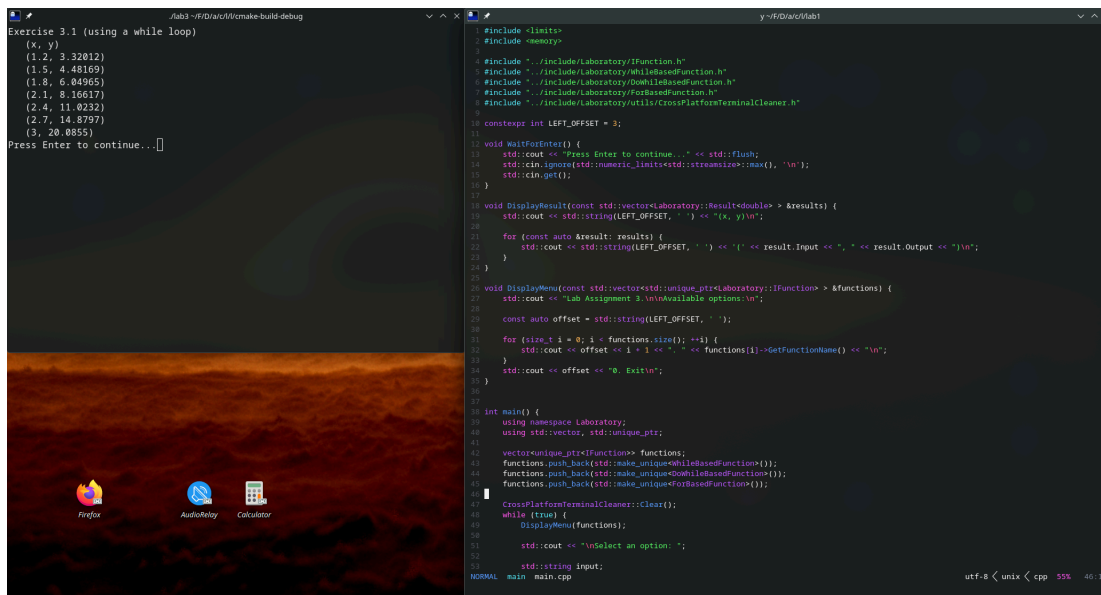


Рис. 7. Блок-схема алгоритма задания 3.3 (цикл for)

Скриншот результата работы программы



```
Exercise 3.1 (using a while loop)
(x, y)
(1.2, 3.32812)
(1.5, 4.48169)
(1.8, 6.04965)
(2.1, 8.10617)
(2.4, 11.0232)
(2.7, 14.8797)
(3, 20.0855)
Press Enter to continue...

#include <limits>
#include <memory>
#include "../include/Laboratory/IFunction.h"
#include "../include/Laboratory/WhileBasedFunction.h"
#include "../include/Laboratory/DoWhileBasedFunction.h"
#include "../include/Laboratory/ForBasedFunction.h"
#include "../include/Laboratory/Utils/CrossPlatformTerminalCleaner.h"

constexpr int LEFT_OFFSET = 3;

void WaitForEnter() {
    std::cout << "Press Enter to continue..." << std::flush;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::cin.get();
}

void DisplayResult(const std::vector<Laboratory::Result<double>> &results) {
    std::cout << std::string(LEFT_OFFSET, ' ') << "(x, y)\n";
    for (const auto &result: results) {
        std::cout << std::string(LEFT_OFFSET, ' ') << "(" << result.Input << ", " << result.Output << ")\n";
    }
}

void DisplayMenu(const std::vector<std::unique_ptr<Laboratory::IFunction>> &functions) {
    std::cout << "Lab Assignment 3.\n\nAvailable options:\n";
    const auto offset = std::string(LEFT_OFFSET, ' ');
    for (size_t i = 0; i < functions.size(); ++i) {
        std::cout << offset << i << " - " << functions[i]->GetFunctionName() << "\n";
    }
    std::cout << offset << "0. Exit\n";
}

int main() {
    using namespace Laboratory;
    using std::vector, std::unique_ptr;

    vector<unique_ptr<IFunction>> functions;
    functions.push_back(std::make_unique<WhileBasedFunction>());
    functions.push_back(std::make_unique<DoWhileBasedFunction>());
    functions.push_back(std::make_unique<ForBasedFunction>());

    CrossPlatformTerminalCleaner::Clear();
    while (true) {
        DisplayMenu(functions);

        std::cout << "\nSelect an option: ";

        std::string input;
        NORMAL main main.cpp utf-8 < unix < cpp 55% 40.1
```

Рис. 8. Результат выполнения программы лабораторной работы №3

Вывод

В ходе лабораторной работы №3 были разработаны три программы на языке C++, реализующие циклические алгоритмы с операторами `while`, `do-while` и `for`. Задания объединены в единый проект с объектно-ориентированной архитектурой на основе абстрактного класса `IFunction`. Полиморфизм обеспечивает единообразное взаимодействие с каждой из реализаций через меню. Все три алгоритма протестированы и дают корректный результат.

Лабораторная работа №4. Одномерный массив

Цель работы

Изучить принципы работы с одномерными массивами в языке C++. Освоить генерацию псевдослучайных чисел стандартными средствами, вывод массива в виде матрицы, а также обработку элементов массива по заданным условиям.

Задание

Написать программу на языке C++ согласно варианту:

- сформировать одномерный массив из K элементов — случайных целых чисел в интервале $[z_1; z_2]$;
- вывести массив на экран в виде матрицы из n строк и m столбцов;
- выполнить для массива задание согласно варианту;
- построить блок-схему к программе.

Таблица 5 — Исходные данные варианта 7

Вариант	K	z_1	z_2	n	m	Задание
7	70	-40	60	10	7	Определить количество и сумму элементов, меньших -10 и больших 30.

Блок-схема программы

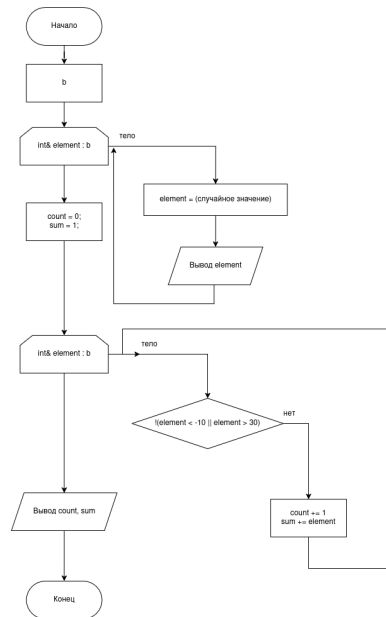


Рис. 9. Блок-схема алгоритма лабораторной работы №4

Листинг программы

Листинг 13 — *main.cpp* (ЛР №4)

```

#include <array>
#include <iomanip>
#include <iostream>
#include <random>

constexpr int ELEMENTS_COUNT_K = 70;
constexpr int ELEMENT_MINIMUM_Z1 = -40;
constexpr int ELEMENT_MAXIMUM_Z2 = 60;
constexpr int ROWS_N = 10;
constexpr int COLUMNS_M = 7;

int main() {
    std::mt19937 rng(std::random_device{}());
    std::uniform_int_distribution dist(ELEMENT_MINIMUM_Z1, ELEMENT_MAXIMUM_Z2);

    std::cout << "Here is an array (represented as a matrix):\n";

    std::array<int, ELEMENTS_COUNT_K> b{};
    int counter = 0;
    for (int& element : b) {
        element = dist(rng);
        std::cout << std::setw(3) << element << ' ';

        counter++;
        if (counter == ROWS_N) {
            std::cout << '\n';
            counter = 0;
        }
    }

    int count = 0;

```

```

int sum = 0;
for (const int& element: b) {
    if (!(element < -10 || element > 30)) {
        continue;
    }

    count += 1;
    sum += element;
}

std::cout << "\nData for elements less than -10 or greater than 30:\n";
std::cout << "Quantity: " << count << '\n';
std::cout << "Sum: " << sum << '\n';

return 0;
}

```

Скриншот результата работы программы

```

~/Files/Development/archive/cpp_1/labs/lab4/cmake-build-debug (main)
./lab4
Here is an array (represented as a matrix):
-25 -10 -32 -16 47 9 39 22 8 -9
-1 -39 -35 42 40 41 -30 -36 39 50
-18 51 -24 38 38 -5 6 -33 -11 10
-29 15 5 30 -9 4 48 16 60 -39
33 38 48 12 7 28 -15 -27 -36 -22
54 -6 -38 -17 -21 45 59 4 7 -16
21 -22 -40 -29 25 -26 -6 -31 50 1

Data for elements less than -10 or greater than 30:
Quantity: 45
Sum: 153
~/Files/Development/archive/cpp_1/labs/lab4/cmake-build-debug (main)

```

Рис. 10. Результат выполнения программы лабораторной работы №4

Вывод

В ходе лабораторной работы №4 была разработана программа на языке C++ для работы с одномерным массивом. Массив из $K = 70$ элементов заполнен случайными целыми числами в диапазоне $[-40; 60]$ с помощью генератора `std::mt19937`. Массив выведен на экран в виде матрицы 10×7 . Для массива определены количество и сумма элементов, удовлетворяющих условию: элемент меньше -10 или больше 30 .

Приложение 1

Все лабораторные работы размещены в открытом репозитории на GitHub и доступны по ссылке:

https://github.com/TimeBean/cpp_lab